Clase 35 · Bagging y Random Forest

Profesor Bastian Galasso · Etapa 7 del roadmap · continúa Árboles de decisión (C34) y precede a Boosting (C36)

La gran idea de la clase

Un árbol profundo es **inestable**: si cambian unas pocas filas, cambian sus preguntas. La solución es entrenar **muchos árboles, cada uno con una muestra distinta** (bootstrap: muestras con reemplazo) y **agregar** sus predicciones por voto o promedio: eso es **bagging**. **Random Forest** agrega un segundo azar: en cada nodo solo se evalúa un subconjunto de variables, así los árboles se parecen menos entre sí. «Si varios modelos cometen errores distintos y luego votan, el error que depende de una muestra particular se reduce.»

Bagging y Random Forest en una imagen



Lo que necesitas ANTES

- Árboles de decisión: preguntas, profundidad, sobreajuste (Clase 34)
- Validación cruzada, AUC, exactitud y RMSE (Clases 30–33)
- Varianza y correlación de variables aleatorias (Etapa 0)
- Pipeline con imputación y one-hot, sin filtrar información al test

Ruta de la clase y datos

- **Inestabilidad → bootstrap y OOB → bagging → varianza del promedio → Random Forest** (`max_features`) → hiperparámetros → límites y usos
- **Heart** (303 pacientes, clasificación AHD) y **Ames Housing** (2.930 casas, regresión del precio)
- Ejemplo de imágenes: el profesor usa CIFAR-10; aquí, los dígitos 8×8 de scikit-learn (sin descargas)
- Pregunta guía: ¿cómo reducir la inestabilidad de un árbol sin perder su capacidad no lineal?

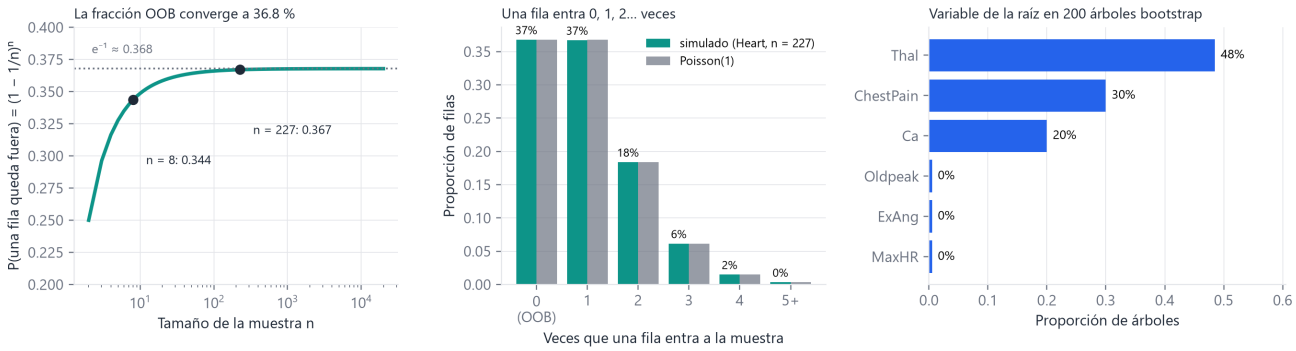
Clase 35 · Árbol inestable, ensamblado y bootstrap

Fuente: transcripción de la clase + diapositivas + notebooks Python del profesor

Un ensamblado combina varios modelos base en una predicción. Para que sirva, los modelos deben ser distintos; el bootstrap fabrica muestras distintas con los mismos datos, sacando n filas con reemplazo.

Conceptos clave

1. **Árboles inestables.** Con otra muestra cambian las variables, los umbrales y hasta la profundidad: bajo sesgo, **alta varianza**. Con 200 muestras bootstrap de Heart la raíz fue Thal en 48.5 %, ChestPain en 30 % y Ca en 20 %.
2. **Ensamblar.** Se entrena un modelo por muestra y una función de agregación entrega la predicción final: voto o promedio de probabilidades en clasificación, promedio en regresión.
3. **Bootstrap.** Se saca una fila, se anota, **se devuelve** y se repite n veces. Las repeticiones crean conjuntos distintos sin recolectar datos nuevos; cada extracción tiene siempre la misma probabilidad.
4. **63.2 % únicas, 36.8 % fuera.** Una fila queda fuera con $(1 - 1/n)^n \to e^{-1} \approx 0.368$. Esas filas son el **OOB** (*out of bag*) de ese árbol.
5. **OOB: validación gratis.** Cada fila se predice solo con los árboles que no la vieron: sirve para estimar el error sin reservar otro conjunto. **No reemplaza** al test final.



Izquierda: fracción OOB teórica. Centro: veces que cada fila entra a una muestra (simulado con Heart, n = 227). Derecha: la raíz de 200 árboles bootstrap.

Fórmulas y reglas

$P(\text{fila fuera}) = (1 - 1/n)^n \to e^{-1} \approx 0.368$

filas únicas esperadas $\approx n \cdot (1 - e^{-1}) \approx 0.632 \cdot n$

veces que entra una fila ~ Poisson(1): 0 → 36.8 %, 1 → 36.8 %, 2 → 18.4 %

bagging: $\hat{y}$ = moda de $h_1 \ldots h_B$ (o promedio de probabilidades)

En Python

```
BaggingClassifier(DecisionTreeClassifier(),
    n_estimators=120, bootstrap=True,
    oob_score=True, n_jobs=-1)
rng.integers(0, n, n) # bootstrap a mano
modelo.oob_score_
```

Con los datos de la clase

- n = 8: 34.4 % fuera · n = 227 (Heart): 36.7 % · n = 2.344 (Ames): 36.8 %
- Simulado con 2.000 muestras: 36.8 % fuera de media (desvío 2.0 puntos)
- Dos árboles bootstrap cualquiera discrepan en 26 % de los pacientes del test
- Un árbol suelto: AUC ≈ 0.73; 100 árboles promediados: ≈ 0.92 (notebook A3)

Cuidado
En clase se dijo «500 de 1.000 registros» por muestra: por defecto son **n** extracciones (≈ 632 filas distintas). Con `max_samples` se cambia. Las muestras se parecen entre sí: no son independientes.

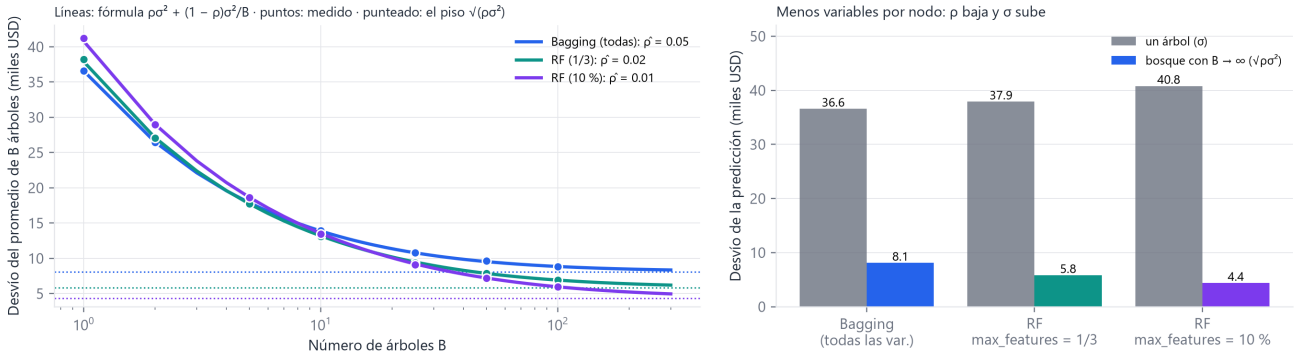
Clase 35 · Bagging y por qué reduce la varianza

Fuente: transcripción de la clase + diapositivas + notebooks Python del profesor

Bagging = bootstrap aggregating: entrena el mismo tipo de modelo en muestras bootstrap distintas y agrega. Reduce la varianza porque promediar suaviza errores individuales, si los árboles no son idénticos.

Conceptos clave

1. **Algoritmo de 4 pasos.** 1) crear B muestras bootstrap · 2) ajustar un árbol en cada una · 3) obtener B respuestas para x · 4) votar o promediar.
2. **Entrenamiento en paralelo.** Cada árbol solo ve su muestra: se reparten entre procesadores (`n_jobs`). En boosting (Clase 36) no se puede: el árbol 2 necesita al 1.
3. **Varianza del promedio.** Con varianza σ^2 por árbol y correlación ρ entre árboles: $\text{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2/B$. Más árboles borran el segundo término; **el primero queda**.
4. **ρ alto = no se gana.** Si todos los árboles se parecen, repiten los mismos errores. Señal en la práctica: un bagging que casi no mejora a un solo árbol.
5. **Cualquier modelo base.** Se puede hacer bagging de una regresión, pero ayuda solo a modelos **inestables** (árboles profundos), no a los estables.



Ames, validación. Se repitió K = 60 veces el entrenamiento con otra submuestra. Los puntos (medidos) caen sobre la fórmula: $\hat{\rho}$ = 0.05 (bagging), 0.02 (RF 1/3), 0.01 (RF 10 %).

Fórmulas y reglas

$\text{Var}(\text{promedio de B}) = \rho\sigma^2 + (1 - \rho)\sigma^2/B$

$B \to \infty$: $\text{Var} \to \rho\sigma^2$ (el piso que no se cruza)

$\rho = 1$: $\text{Var} = \sigma^2$ · $\rho = 0$: $\text{Var} = \sigma^2/B$

supone varianzas iguales y ρ común (nota de la diapositiva 9)

En Python

```
BaggingRegressor(DecisionTreeRegressor(),
    n_estimators=300, n_jobs=-1)
RandomForestRegressor(n_estimators=300,
    n_jobs=-1, random_state=42)
np.mean([a.predict(X) for a in arboles], 0)
```

Con los datos de la clase

- σ (un árbol) ≈ 36.6 mil USD; con B = 100 el promedio de bagging baja a ≈ 8.8 mil (piso $\sqrt{(\rho\sigma^2)}$ ≈ 8.1 mil)
- Fórmula y medición coinciden casi al dólar (ej.: B = 10 → 13.9 mil medido, 13.9 mil fórmula)
- Ames, 300 árboles: un árbol 36.1k · bagging 24.4k · Random Forest (1/3) 23.1k (RMSE en validación)
- Heart: bagging de árboles AUC 0.893 (CV 5×3) frente a 0.753 de un árbol

Cuidado

En clase se dijo «ρ es la correlación entre errores y no se mide». Es la correlación entre las **predicciones** de dos árboles bajo el azar de datos y algoritmo, y sí se estima (notebook B2).

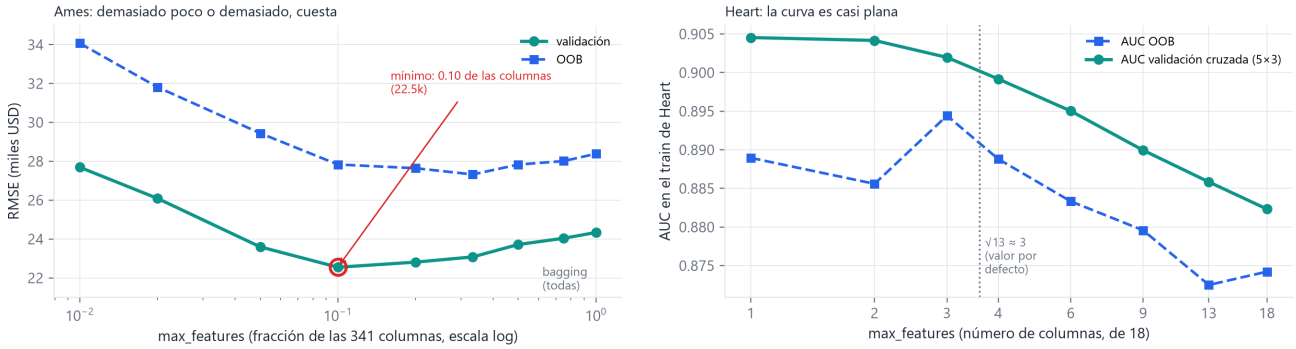
Clase 35 · Random Forest y max_features

Fuente: transcripción de la clase + diapositivas + notebooks Python del profesor

Random Forest = bagging de árboles + selección aleatoria de variables en cada nodo. Si una variable domina, no puede mandar en todos los árboles: los árboles se parecen menos (ρ baja) y el promedio mejora.

Conceptos clave

1. **Dos fuentes de azar.** Filas: cada árbol recibe una muestra bootstrap. Variables: en cada **nodo** solo se evalúan *mtry* = `max_features` candidatas al azar entre las p disponibles.
2. **Por nodo, no por árbol.** El mejor corte se busca **dentro del subconjunto** sorteado en ese nodo; en el siguiente nodo se sortea otro. (La nota de la diapositiva 10 lo dice; en clase se explicó por árbol.)
3. **Qué se compra y qué cuesta.** Menos variables por nodo = menor ρ, pero cada árbol es algo peor (σ^2 mayor, algo más de sesgo). El óptimo es intermedio y se **valida**.
4. **Valores de partida.** Clasificación: `'sqrt'`. Regresión: probar 0.1–0.5 (el RF de regresión de scikit-learn usa 1.0 por defecto, es decir, **un bagging**).
5. **Bagging vs Random Forest.** Mismo bootstrap, misma agregación y OOB en ambos; la diferencia clave es `max_features`. «Un parámetro para controlar la similitud entre árboles.»



Ames (izquierda): RMSE de validación y OOB al variar max_features con 300 árboles. Heart (derecha): AUC en el train, OOB y validación cruzada 5×3.

Fórmulas y reglas

candidatas por nodo = max_features

'sqrt' → √p · 'log2' → $\log_2 p$ · fracción → □f·p□

P(una variable dada es candidata en un nodo) = m/p

m = p □ Random Forest = bagging

En Python

```
RandomForestClassifier(n_estimators=300,
    max_features='sqrt', oob_score=True)
RandomForestRegressor(max_features=0.2)
# por árbol (otra cosa): BaggingRegressor(
# max_features=0.1)
```

Con los datos de la clase

- Ames (341 columnas tras el one-hot): 1 % de columnas → RMSE 27.7k · 10 % → 22.5k (mínimo) · 1/3 → 23.1k · todas (bagging) → 24.3k
- Mismo 10 %: por nodo (RF) 22.5k vs por árbol (random subspace) 27.1k
- Heart (18 columnas): m = 1 → AUC 0.905 · m = 3 → 0.902 · m = 18 → 0.882; desvío ≈ 0.03: curva casi plana
- Con el one-hot, 13 variables son 18 columnas: √18 ≈ 4 (en clase: √13 ≈ 3)

Cuidado
«Con max_features = 1 salen modelos muchísimo mejores» (clase): en Heart es indistinguible; en Ames es lo **peor** del barrido. Se valida, no se recita.

Clase 35 · OOB, número de árboles e hiperparámetros

Fuente: transcripción de la clase + diapositivas + notebooks Python del profesor

Más árboles estabilizan el resultado y cuestan tiempo y memoria; el OOB estima el error con un solo ajuste; la complejidad de cada árbol y el muestreo se afinan con validación cruzada, nunca mirando el test.

Conceptos clave

1. **n_estimators.** Más árboles bajan el término $(1 - \rho)\sigma^2/B$ y estabilizan, **sin sobreajustar**; el costo sube casi lineal. La mejora se aplana pronto.
2. **max_features.** Menos variables = menor correlación y quizá algo más de sesgo. Se prioriza junto con n_estimators y el tamaño mínimo de hoja.
3. **max_depth / min_samples_leaf.** Controlan la complejidad de cada árbol. Con hojas de 1 observación el bosque memoriza el train (exactitud 1.000) pero igual generaliza.
4. **bootstrap / max_samples / class_weight.** Definen si hay reemplazo y cuántas filas ve cada árbol; `class_weight` ayuda con una clase rara (p. ej. 2 % de positivos).
5. **Búsqueda y OOB.** `GridSearchCV` prueba una grilla; `RandomizedSearchCV`, n_iter combinaciones. El OOB sirve para comparar configuraciones; el test queda para el final.



Ames, validación (hold-out). Izquierda: error al agregar árboles. Derecha: el OOB (sin validación aparte) se estabiliza igual, pero más alto: usa ≈ 37 % de los árboles.

Fórmulas y reglas

grilla: nº combinaciones × nº bloques = ajustes

24 comb. × 5 bloques = 120 ajustes por modelo

tiempo en paralelo ≥ tiempo serie / nº de núcleos

OOB: cada fila usa ≈ 0.368·B árboles

En Python

```
RandomForestClassifier(oob_score=True)
rf.oob_score_ · rf.oob_decision_function_
GridSearchCV(pipe, grilla, cv=5,
    scoring='roc_auc', n_jobs=-1)
RandomizedSearchCV(pipe, dist, n_iter=12)
```

Con los datos de la clase

- Ames: a las 34 árboles el RF queda a menos de 2 % de su valor final (RMSE 23.1k con 100 y con 300 árboles)
- OOB vs CV vs test: Heart 0.811 · 0.798 · 0.855 (exactitud); Ames 26.0k · 26.3k · 27.0k. OOB: 1 ajuste; CV: 5
- Heart: RF base AUC CV 0.899; tras Grid o Random, ≈ 0.911: la búsqueda ayuda ≈ 0.01
- n_jobs = −1 acelera ≈ 6.6× con 16 núcleos lógicos (no 16×)

Cuidado
El OOB es levemente pesimista con pocos árboles y **no reemplaza** al test independiente. Ajustar mirando el test repetidamente destruye su validez. Incluye tu configuración base en la grilla.

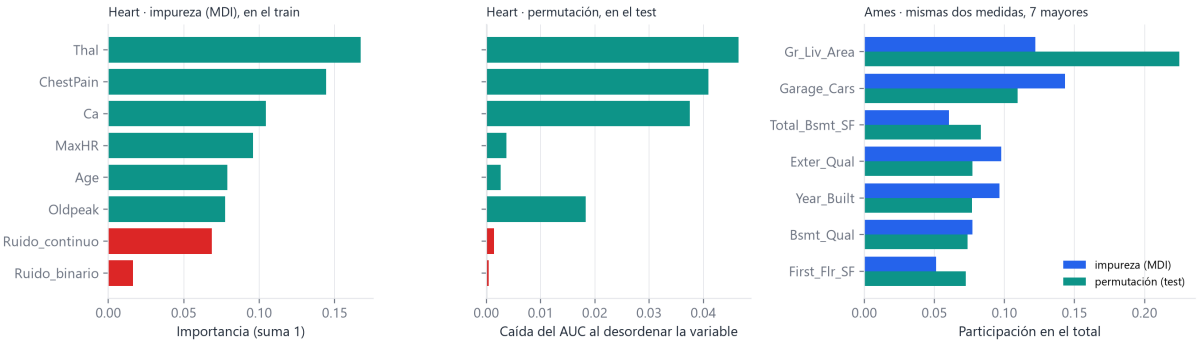
Clase 35 · Importancia, límites y aplicaciones

Fuente: transcripción de la clase + diapositivas + notebooks Python del profesor

Rinde en datos tabulares no lineales, sin escalar variables; pierde interpretabilidad, no extrapola en regresión y no ve estructura espacial. La importancia por impureza puede engañar: prefiere la de permutación.

Conceptos clave

1. **Ventajas.** Buen desempeño sin suponer linealidad, menos varianza que un árbol, interacciones y umbrales automáticos, poco sensible a la escala, OOB incluido.
2. **Limitaciones.** Menos interpretable, más memoria y tiempo de predicción, **no extrapola** fuera del rango visto y pide variables numéricas (codificar las categóricas).
3. **Importancia por impureza (MDI).** Se calcula en el **train** y favorece variables con muchos cortes posibles, aunque sean ruido. Mejor: **permutación** sobre datos no vistos. Ninguna es causalidad.
4. **Flujo recomendado.** 1) separar test · 2) elegir la métrica (clases raras: recall, F1, PR-AUC) · 3) ajustar con CV u OOB · 4) evaluar una vez y explicar.
5. **Dónde encaja.** Riesgo y fraude, salud, operaciones, marketing. En imágenes, texto o series muy largas: modelos especializados.



Heart con dos columnas de ruido puro agregadas (rojo): la impureza (train) premia al ruido continuo; la permutación (test) lo deja en ≈ 0. A la derecha, las dos medidas en Ames.

Fórmulas y reglas

MDI = Σ (N_t/N)·ΔGini en los nodos de la variable

perm = métrica_base − métrica_con_la_variable_desordenada

RF regresión: ŷ queda entre el mín y el máx de y (train)

En Python

```
permutation_importance(pipe, X_test, y_test,
    scoring='roc_auc', n_repeats=30)
rf.feature_importances_ # MDI (train)
from sklearn.metrics import recall_score,
    f1_score, average_precision_score
```

Con los datos de la clase

- Ruido continuo: puesto 7 de 15 por impureza (6.9 %, sobre Chol, RestBP, Sex…) puesto 10 por permutación (≈ 0.001 de AUC)
- Ames: Gr_Liv_Area vale 22.5 % por permutación (+11.9 mil USD de RMSE) y 12.2 % por impureza
- Entrenado con casas < 1.800 pies², el bosque no pasa de ≈ 158 mil USD en las > 2.400 pies²

Cuidado
Con 76 pacientes y 5 repeticiones (notebook del profesor) la permutación es ruidosa. En clase se dijo que la impureza engaña «porque hay menos variables»: la causa es el sesgo hacia variables con muchos cortes.

Chuleta, hiperparámetros y checklist

Lo que vas a ajustar en cualquier Random Forest

Hiperparámetro	Qué controla	Efecto si lo subes	Valor de partida
n_estimators	Número de árboles (B)	estabiliza; más tiempo y memoria; se aplana pronto	300–500
max_features	Variables candidatas por nodo	mayor ρ; menos sesgo individual (1.0 = bagging)	'sqrt' (clasif.); 0.1–0.5 (regr.)
max_depth	Profundidad máxima de cada árbol	más complejidad; riesgo de memorizar	None o validar 6–20
min_samples_leaf	Observaciones mínimas por hoja	hojas y predicciones más suaves	1–5
bootstrap · max_samples	Muestreo de filas (con reemplazo, fracción)	max_samples menor: más rápido y diverso	True · None (n filas)
class_weight	Peso de cada clase	da más peso a la clase rara	None; 'balanced' si hay desbalance
oob_score · n_jobs	Error interno · paralelismo	OOB gratis · acelera (no linealmente)	True · −1

Hiperparámetros: diapositiva 14 (nombres de la API de scikit-learn). Los valores de partida son orientativos; el profesor pide validarlos con CV u OOB y una métrica alineada con el problema.

Modelo	Datos de entrenamiento	Varianza	Interpretable	Paralelizable
Árbol de decisión	Todos	Alta	Sí	—
Bagging de árboles	Muestras bootstrap	Baja	No	Sí, entre árboles
Random Forest	Bootstrap + variables al azar por nodo	Más baja (menor ρ)	No	Sí, entre árboles
Boosting (Clase 36)	Todos, con pesos o residuos	—, baja sesgo	No	No entre árboles

Checklist para entrenar un Random Forest

	Paso	Cómo sé que lo hice bien
☐	Parte train/test (estratificado si es clasificación)	en regresión no se estratifica
☐	Preprocesa dentro del Pipeline	imputación y one-hot se aprenden solo con el train; el RF necesita variables numéricas
☐	Elige la métrica antes de ajustar	con clases raras no uses exactitud
☐	Compara con un árbol y con un bagging	para ver cuánto aportan max_features y el promedio
☐	Valida max_features, n_estimators y min_samples_leaf	CV u OOB; el test no se toca
☐	Mira train vs OOB/CV vs test	una brecha grande no es mala si el error de test es el menor
☐	Interpreta con permutación sobre datos no vistos	la impureza (MDI) puede engañar
☐	Reporta media ± desvío	con 76 pacientes de test una sola partición no basta

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales, preguntas clave y cuándo pasar a la siguiente clase

Alertas del material

Diapositivas y clase (verificados con código)

- **«Cada árbol toma una muestra de las variables (la 1, 5 y 8; el otro la 7, 15 y 20)»** (clase): la selección es **por nodo**, como dice la nota de la diapositiva 10. Por árbol sería *random subspace* y rinde mucho peor (27.1k vs 22.5k de RMSE).
- **«De 1.000 registros se toman 500»**: la muestra bootstrap tiene n extracciones por defecto. **«Las muestras son independientes»**: se parecen entre sí.
- **«max_features = 1 da modelos muchísimo mejores»**: en Heart es indistinguible; en Ames es lo peor. **«Casi elimina el sobreajuste»**: con hojas de 1 el bosque llega a exactitud 1.000 en el train.
- **«Con 10 procesadores, 10 veces menos tiempo»** es el límite ideal (real ≈ 6.6× con 16 núcleos). **«√13 ≈ 3 variables»**: con el one-hot son 18 columnas (√18 ≈ 4).
- **«Los bosques aceptan categóricas»**: en scikit-learn hay que codificarlas. **«Boosting pierde el OOB»**: con `subsample < 1` existe `oob_improvement_`. **«AUC siempre > 0.5»**: puede ser < 0.5 si ordena al revés.
- **Diapositiva 9**: ρ es la correlación entre **predicciones**, no «errores». **Diapositiva 13**: scikit-learn promedia probabilidades (≠ voto con hojas > 1).

Notebooks del profesor

- `RandomForestRegressor(max_features=1.0)` **es un bagging**: casi idéntico al `BaggingRegressor` (RMSE 29.1k vs 29.3k); con 1/3 baja a 26.7k.
- `class_weight='balanced'` solo en el bosque; y en la búsqueda aleatoria el «Bagging» puede sortear `estimator__max_features`, lo que lo convierte en un Random Forest.
- Permutación sobre 76 pacientes con 5 repeticiones: ruidosa. Ruta de datos fija y `FAST_MODE = False` (en clase se recomendó True).
- **Imágenes** usa CIFAR-10 (descarga ≈ 170 MB) y **no se vio en clase**; aquí se reemplaza por los dígitos de scikit-learn. Rinde bien con píxeles reordenados al azar (0.973) pero cae a 0.504 si se desplaza la imagen 1 píxel: no entiende la posición.

Transcripción con ruido

«busstrap/boost trap» (bootstrap), «Vallin/Bind/Baguí» (bagging), «random forks/Randolph Farris» (Random Forest), «obb/OVE/auto bag» (OOB), «rock aoc» (ROC AUC), «aquira» (accuracy), «ricó» (recall), «min sample lift» (`min_samples_leaf`), «her.xlsx» (Heart.xlsx). Bucle de Whisper en las líneas 231–252; comentarios ajenos en 319–329, 453 y 840–848. La clase termina antes de Ames y de las imágenes.

Preguntas de repaso

#	Pregunta	Respuesta corta
1	¿Por qué aparecen filas repetidas en el bootstrap?	Se extrae con reemplazo; con n grande ≈ 63.2 % únicas y ≈ 36.8 % OOB.
2	¿Qué problema busca resolver max_features?	Reducir la correlación ρ entre árboles, el piso $\rho\sigma^2$ de la varianza.

Criterio de dominio para pasar a la siguiente clase

- ☐ Calculo a mano la probabilidad OOB $(1 - 1/n)^n$ y leo una muestra bootstrap
- ☐ Aplico $\rho\sigma^2 + (1 - \rho)\sigma^2/B$ y explico por qué $B \to \infty$ no basta
- ☐ Distingo bagging (todas las variables) de Random Forest (subconjunto por nodo)
- ☐ Calculo a mano el OOB, un voto y una importancia por impureza
- ☐ Elijo max_features, n_estimators y min_samples_leaf con CV u OOB, sin tocar el test
- ☐ Resuelvo los bloques 1 a 9 del cuaderno con al menos 80% de aciertos